Trabajo Práctico N°3 - Sistemas de Inteligencia Artificial

Perceptrón Simple y Multicapa

Alicia Cobo Iglesias · Lucas Di Candia · Axel Farina · Emilio Mitchell · Tomas Pietravallo

Tabla de contenidos

01 Ejercicio 1: detección de fraude

02 Ejercicio 2: dígitos con MLP

03 Ejercicio 3: más datos y objetivo del 98%

04 Opcionales: ReLU, features e interpretabilidad

05 Conclusiones

Implementación en Rust

00. Herramientas

Core propio, sin frameworks

Perceptrón escalón, lineal y no lineal, y perceptrón multicapa con backpropagation explícita. Inicialización Xavier y He.

Operaciones matriciales

El mini-batch se entrena como productos de matrices y se reparte en hilos con rayon. El gradiente analítico se verifica contra diferencias finitas.

Configuración y persistencia

Cada candidato se define en TOML. Los modelos se guardan con su configuración, y continue, score y evaluate los levantan.

Progreso y análisis separados

Logs por candidato y dashboard en vivo en un proceso aparte. Cada corrida guarda CSVs, y los gráficos se derivan de ellos. Los opcionales suman calibrate, feature-study, noise y attribution.

Ejercicio 1

Fraude

Detección de fraude con un perceptrón simple

Detección de fraudes

01. Ejercicio 1

Objetivo del proyecto

- Estimar la probabilidad de fraude de una transacción online.
- Replicar las predicciones del BigModel, que es caro de usar.
- Usar una única neurona: perceptrón lineal y no lineal.
- Es Knowledge Distillation: el BigModel enseña al TinyModel.

Datos de entrada

7.500 transacciones con 9 features. Quedan 8 entradas y un sesgo: 9 parámetros en total.

Target a predecir

Probabilidad continua en [0, 1] del BigModel. flagged_fraud no se usa para entrenar: solo evalúa y fija el umbral.

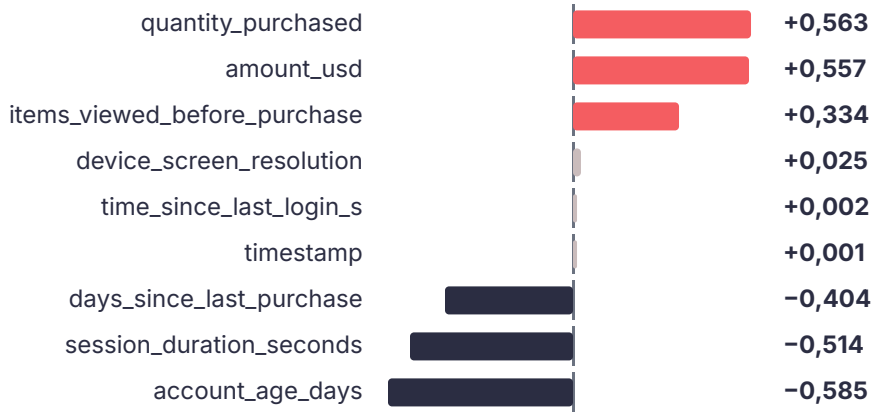
Análisis de datos

01. Ejercicio 1

Hallazgos y preprocesamiento

- 7.500 muestras, 11,6% de fraude: hay desbalance.
- Sin faltantes ni filas duplicadas.
- log1p en 3 columnas con asimetría mayor a 2.
- z-score en todas las features.
- timestamp se descarta: correlación de 0,001.

Correlación de Pearson con el BigModel



Coral: correlación positiva. Azul oscuro: negativa. Gris: casi nula.

Perceptrón simple: lineal vs no lineal

01. Ejercicio 1

MSE contra el BigModel, con todas las muestras



Cada modelo usa la misma grilla de tasas de aprendizaje y se queda con la mejor. El eje parte de cero.

−25%

de MSE del sigmoide frente al lineal, con las mismas 9 entradas.

Salidas fuera de [0, 1]

Lineal: 7,6% de sus salidas. El sigmoide queda en [0,006; 0,999].

Estudio de aprendizaje

01. Ejercicio 1

a) ¿Underfitting?

Sí, sobre todo en el lineal.

Su error es un 33% mayor que el del sigmoide sobre los mismos datos de entrenamiento, y la loss se aplana en un nivel alto. Un hiperplano no reproduce la forma de S de la probabilidad.

b) ¿Saturación?

Sí, en dos sentidos.

Los dos llegan a un plateau con error residual: una neurona no captura interacciones entre features. Además, el 3,7% de las salidas del sigmoide queda en los extremos, donde el gradiente casi desaparece.

c) ¿Cuál elegimos?

El perceptrón sigmoide.

Tiene el menor error y su salida está en [0, 1] por construcción, así que se lee como probabilidad sin recortarla. Es el que pasa al estudio de generalización.

Generalización: test aparte y K-Fold (K=5)

01. Ejercicio 1

Desarrollo: 85% (6.374 filas), validación cruzada estratificada · Test: 15% (1.126 filas), se consulta una sola vez

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Test 1.126

Sin fuga de datos

Cada fold ajusta su propio scaler con sus filas de entrenamiento. Todos los folds tienen entre 11,1% y 11,8% de fraude.

Qué se elige con los folds

La tasa de aprendizaje (0,05) y las épocas (76, mediana del early stopping). Después se entrena con todo el 85%.

¿Mejor conjunto de train?

Ninguno: elegir el split que mejor da sobreajusta a la partición. Con K-Fold cada fila valida una vez y se informa media ± desvío.

Métricas de evaluación del modelo

01. Ejercicio 1

Fidelidad

MSE, RMSE y R^2 contra el BigModel: qué tan bien lo imita.

Uso: mide la destilación.

Precisión

De lo que marcamos como fraude, cuánto lo es de verdad.

Uso: evita revisiones inútiles y fricción con clientes legítimos.

Recall

De los fraudes que existen, cuántos detectamos. **Uso:** mide el fraude que se nos escapa, que suele costar más.

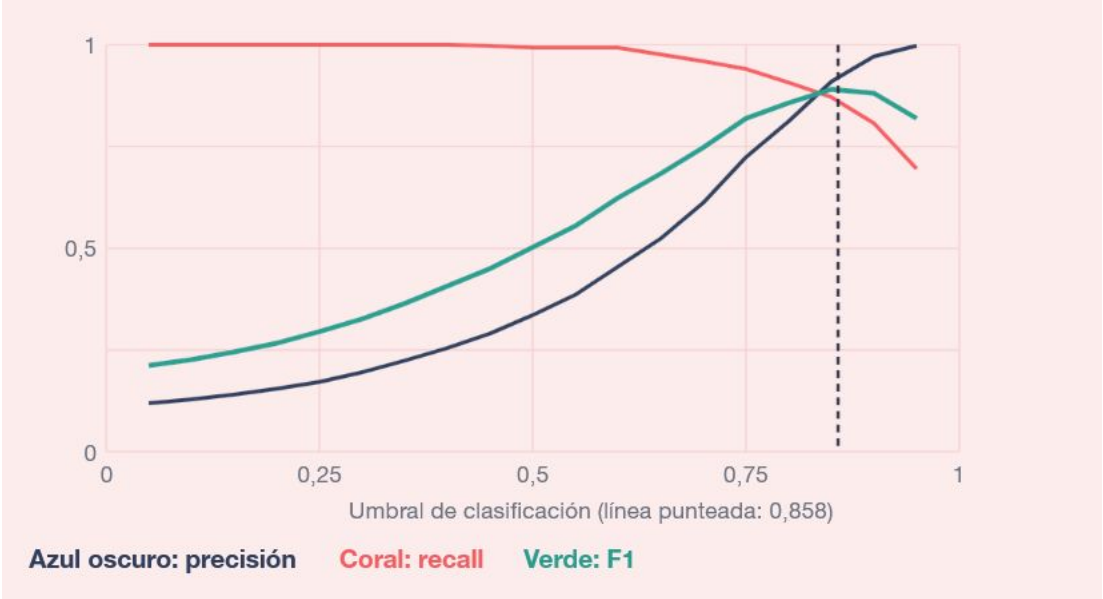
F1 y AP

F1 es la media armónica de precisión y recall. AP es el área bajo la curva precisión-recall. **Uso:** F1 elige el umbral.

Por qué no accuracy: un modelo que nunca marca fraude ya tiene 88,4% de accuracy. Se informa, pero no decide.

Elección del umbral óptimo

01. Ejercicio 1



Predicciones out-of-fold evaluadas contra flagged_fraud para cada umbral θ.

θ = 0,858

Umbral elegido

Maximiza F1. Ante un empate gana el mayor recall. Coincide con el corte natural del BigModel (0,85).

Mejor modelo y recomendación

01. Ejercicio 1 · resultado en test (1.126 transacciones)

0,858	96,5%	80,9%	0,880
Umbral	Precisión	Recall	F1

110 fraudes detectados	**26** fraudes que se escapan
4 falsas alarmas	**986** legítimas aprobadas

Qué le recomendamos a CompanyX

- Un perceptrón sigmoide de 9 parámetros, con R^2 de 0,828 contra el BigModel.
- Operar con $\theta = 0{,}858$ como punto de partida.
- Si un fraude cuesta mucho más que una revisión: calibrar y usar el umbral por costo (ver Opcionales).

Métricas: validación cruzada vs test

01. Ejercicio 1 · umbral 0,858, TinyModel sigmoide

Métrica	5 folds (media ± desvío)	Test
MSE vs BigModel	0,0164 ± 0,0006	0,0157
R^2 vs BigModel	0,821 ± 0,006	0,828
Precisión	0,929 ± 0,013	0,965
Recall	0,869 ± 0,013	0,809
F1	0,898 ± 0,007	0,880
Average precision	0,957 ± 0,005	0,950
Accuracy	0,977 ± 0,001	0,973

Desvíos chicos

El F1 varía ±0,007 entre folds: el resultado no depende de un split afortunado.

Precisión y recall en test

La precisión sube (0,965) y el recall baja (0,809). El test tiene solo 136 fraudes, así que el recall es menos estable.

Conclusiones

01. Ejercicio 1

La activación importa

El sigmoide no curva la frontera de decisión, pero al comprimir la salida a [0, 1] baja el error un 25% frente a un objetivo probabilístico.

El umbral es una decisión de negocio

Con clases desbalanceadas la accuracy engaña. Fijar θ con F1 traduce la estadística en costo operativo.

Las features compensan la capacidad

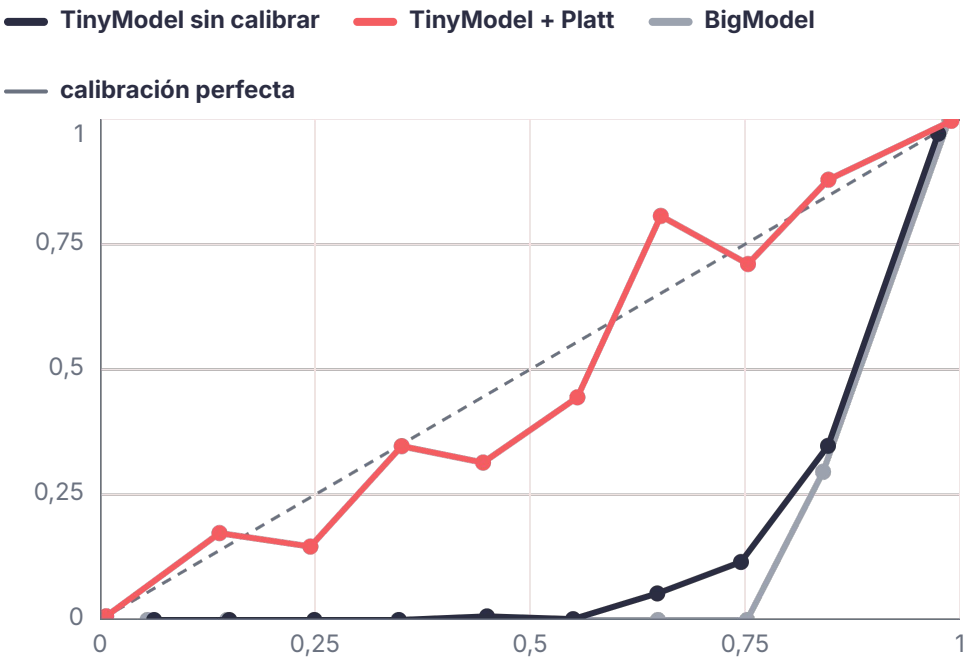
Sin capas ocultas, la no linealidad debe entrar por el preprocesamiento: log1p en las colas y descartar timestamp.

Knowledge Distillation

Con 9 parámetros, el TinyModel llega a F1 de 0,880 en test y reproduce al BigModel con R^2 de 0,828.

Calibración: ¿el score es una probabilidad?

01. Ejercicio 1 · opcional · diagrama de confiabilidad (predicciones out-of-fold, 10 bins)



Eje x: probabilidad media predicha en el bin. Eje y: tasa real de fraude (flagged_fraud).

0,297 → 0,011
ECE en test, sin calibrar → Platt

0,146 → 0,022
Brier en test, sin calibrar → Platt

El ranking no cambia

Average precision 0,950 antes y después: Platt es monótona. El BigModel tampoco está calibrado (ECE 0,304).

Calibración: por qué y para qué

01. Ejercicio 1 · opcional · test, un fraude no detectado cuesta 20 revisiones

El TinyModel imita, no estima

Se entrena con MSE contra el BigModel, que separa perfecto en 0,85 pero no es una probabilidad: su score medio es 0,42 y la tasa real de fraude, 0,12.

Platt: σ(a · logit(s) + b)

a = 2,68 y b = −5,08, ajustados con predicciones out-of-fold. Equivale a reescalar los pesos de la misma neurona: siguen siendo 9 parámetros.

Umbral por costo

Con probabilidades calibradas, conviene marcar si p > 1/(1 + 20) ≈ 0,048. Sin calibrar, esa fórmula no sirve.

Política	Umbral	TP	FP	FN	Recall	Costo
Sin calibrar, umbral F1 (actual)	0,858	110	4	26	0,809	524
Sin calibrar, umbral de costo	0,048	136	962	0	1,000	962
Platt, umbral de costo	0,048	130	84	6	0,956	204
Isotónica, umbral de costo	0,048	132	124	4	0,971	204

Resultado: con Platt, el umbral teórico cuesta 204 contra 524 del umbral F1: 2,6 veces menos, sin buscar el umbral a mano. Costo = FP + 20 · FN.

Ejercicio 2

Dígitos

Clasificación con un perceptrón multicapa

Detección de dígitos (OCR)

02. Ejercicio 2

Cómo evaluamos el desempeño

- digits.csv se parte 80/20, estratificado por clase, y ahí se ajustan parámetros e hiperparámetros.
- El candidato ganador se elige por accuracy de validación; las épocas, por early stopping.
- Se reentrena con el 100% de digits.csv y digits_test.csv se usa una sola vez, como producción.
- Métricas: cross-entropy, accuracy, recall por clase y matriz de confusión.

Datos

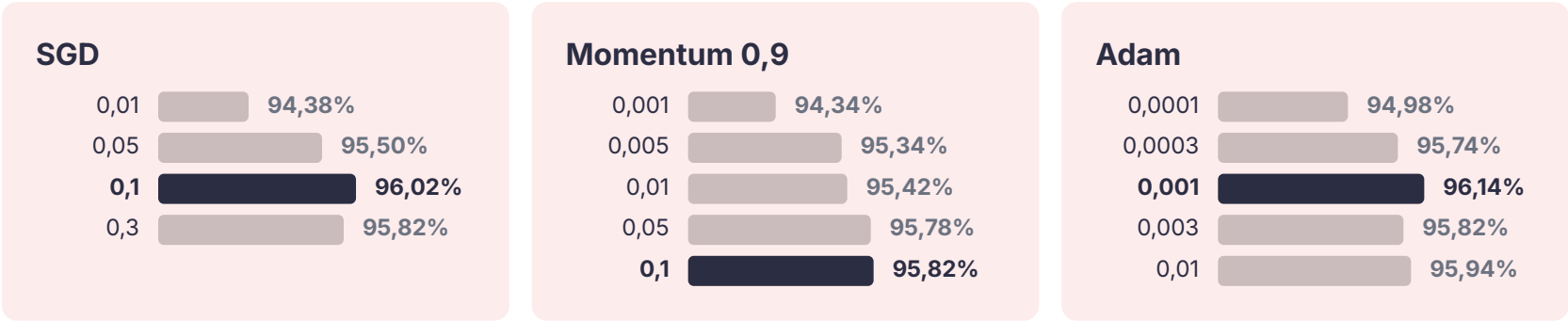
Imágenes de 784 píxeles. Aprendizaje con 12.449 filas y producción con 2.497. Salida de 10 clases con softmax.

Variantes exploradas

26 candidatos: un factor a la vez alrededor de un modelo de referencia. Tasa de aprendizaje, arquitectura y optimizador como mínimo, más activación, batch e inicialización.

Impacto de la tasa de aprendizaje

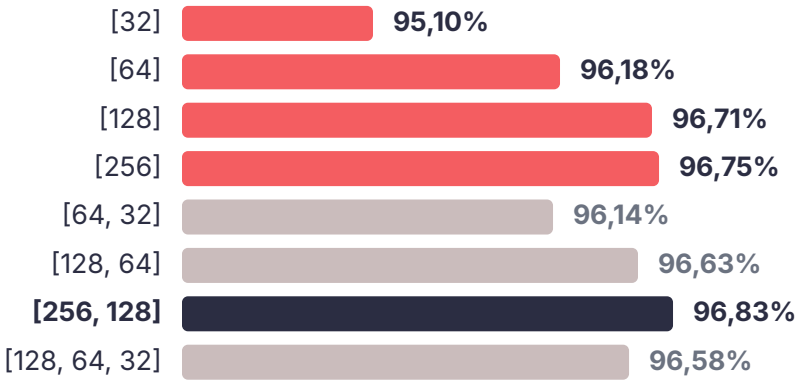
02. Ejercicio 2 · accuracy de validación, eje desde 93%



Lectura: la barra azul oscuro es la mejor tasa de cada optimizador. Una tasa muy baja no converge en 40 épocas; una alta converge rápido pero se estanca peor. Momentum multiplica la tasa efectiva por ~10.

Impacto de la arquitectura

02. Ejercicio 2 · Adam 0,001 · eje desde 94%



Coral: una capa oculta. Gris: varias capas. Azul oscuro: la mejor.

Análisis

- El ancho importa más que la profundidad: de 32 a 128 neuronas suma 1,6 pp.
- Una tercera capa oculta no mejora.
- Salvo [32], todas superan 99% en train: el límite es la generalización, no la capacidad.

Impacto del mecanismo de optimización

02. Ejercicio 2 · cada uno con su mejor tasa

SGD

96,02%

tasa 0,1 · mejor época 15

Momentum (β = 0,9)

95,82%

tasa 0,1 · mejor época 6

Adam

96,14%

tasa 0,001 · mejor época 12

Con una buena tasa, empatan

Los tres quedan a 0,3 pp entre sí. Comparar optimizadores con una misma tasa fija habría medido la tasa, no el optimizador.

La diferencia es la robustez

Adam se mantiene sobre 95,7% en todo el rango 0,0003 a 0,01. SGD pierde 1,6 pp si la tasa se aleja de su óptimo.

Otros hiperparámetros

02. Ejercicio 2 · modelo de referencia [64, 32], Adam 0,001

Factor	Variante	Validación	Nota
Activación	ReLU	96,14%	12 épocas
Activación	Sigmoide	96,18%	35 épocas
Activación	Tanh	95,78%	—
Batch	16	96,06%	—
Batch	64	96,14%	referencia
Batch	256	95,62%	menos pasos
Inicialización	Xavier	96,14%	referencia
Inicialización	He	96,18%	—

Lectura

- Casi no hay diferencia entre activaciones: el sigmoide iguala a ReLU pero tarda 3 veces más en converger.
- Lo que más pierde es batch 256 (−0,5 pp), por hacer menos actualizaciones por época; tanh pierde 0,4 pp.
- He contra Xavier es marginal con esta profundidad.

Modelo elegido vs dataset productivo

02. Ejercicio 2 · [784, 256, 128, 10], ReLU, Adam 0,001, 6 épocas

96,83%
Validación

86,86%
digits_test.csv

Una caída de 10 puntos

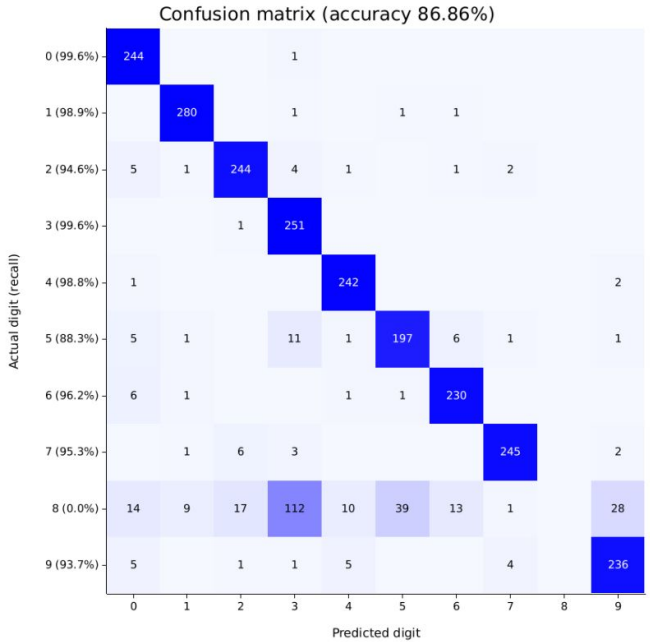
El test tiene 243 ochos (9,7% de las muestras) y el modelo nunca vio uno, porque digits.csv no tiene ninguno.

El techo es cerca de 90%

Ninguna variante de hiperparámetros puede corregir una clase ausente del entrenamiento. Por eso el estudio no lo resuelve.

Matriz de confusión en test

02. Ejercicio 2 · accuracy 86,86%



El 8 nunca se acierta

Sus 243 muestras se reparten entre el 3 (112), el 5 (39), el 9 (28) y el resto. La columna del 8 está vacía: el modelo no puede predecir una clase que no vio.

El resto funciona

Sin el 8, el recall va de 88,3% (el 5) a 99,6%. El techo alcanzable es 90,3%.

Métricas: train vs validación

02. Ejercicio 2 · accuracy en la mejor época de cada candidato

Arquitectura	Optimizador	Train	Valid.	Brecha	Época
[64, 32]	SGD 0,1	99,16%	96,02%	3,13 pp	15
[64, 32]	Momentum 0,1	98,58%	95,82%	2,76 pp	6
[64, 32]	Adam 0,001	99,55%	96,14%	3,41 pp	12
[128]	Adam 0,001	99,82%	96,71%	3,11 pp	12
[128, 64, 32]	Adam 0,001	99,33%	96,58%	2,74 pp	7
[256, 128] elegido	Adam 0,001	99,60%	96,83%	2,77 pp	6

Brecha de unos 3 pp

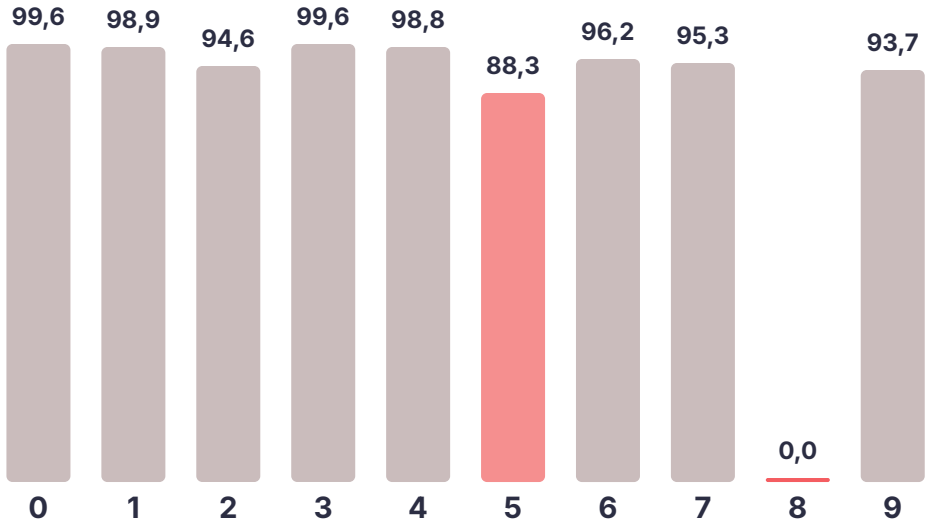
Todos aprenden el train casi por completo; lo que limita es generalizar, no la capacidad.

Modelo elegido en test

Accuracy 86,86% y loss 1,645 sobre 2.497 muestras. Sin el 8, el techo es 90,3%.

Modelo elegido: recall por clase

02. Ejercicio 2 · evaluado en digits_test.csv



Recall en % por dígito.

El dígito 8

- Recall 0%: nunca se predice bien.
- Se confunde sobre todo con el 3 (112 de 243), el 5 y el 9.
- El 5 es la otra clase débil: solo 271 ejemplos en el train.
- Sin el 8, el recall va de 88% a 99,6%.

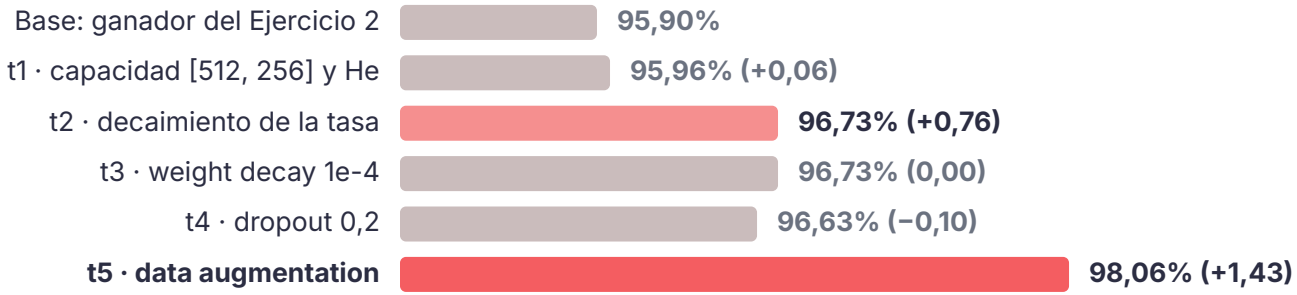
Ejercicio 3

98%

Más datos y regularización

Escalera de técnicas

03. Ejercicio 3 · validación con more_digits.csv, eje desde 95%



Augmentation decide: traslaciones de ±2 px y rotaciones de ±10°. Cada época ve una variante distinta de cada imagen.

Capacidad, weight decay y dropout no aportan solos: la red ya llega a 99,6% en train, así que el problema es la varianza.

Qué cambió en los datos

03. Ejercicio 3 · digits.csv contra more_digits.csv

Filas

12.449 → 15.741

12.052 imágenes son nuevas; 3.689 ya estaban en digits.csv.

Dígito 8

0 → 585

Aparece la clase que faltaba en el Ejercicio 2.

Dígito 5

271 → 542

La clase minoritaria duplica sus ejemplos.

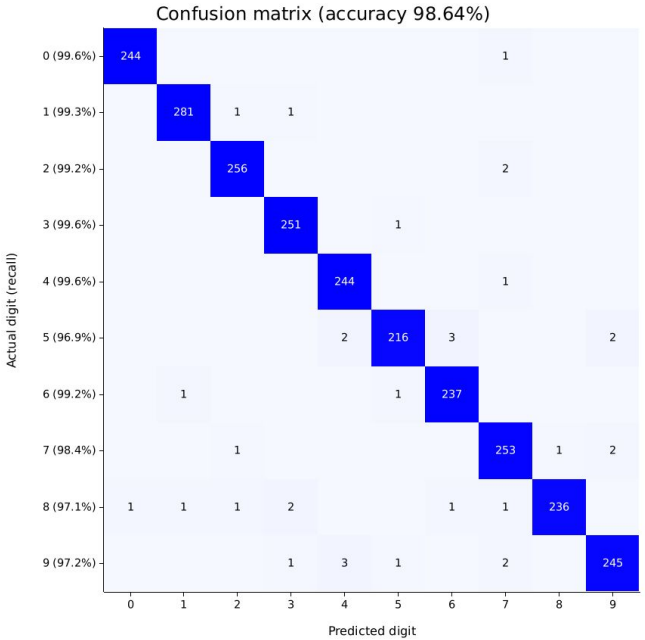
Resto de las clases

+20% cada una

Más variedad de estilos de escritura en cada dígito.

Matriz de confusión en test

03. Ejercicio 3 · accuracy 98,64%



El 8 ya se reconoce

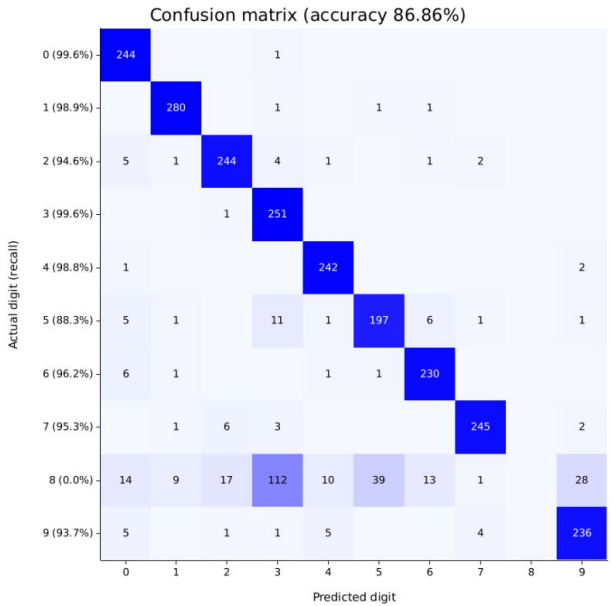
Recall del 8: de 0% a 97,1% (236 de 243). Los errores que quedan están dispersos: ninguna confusión supera 3 muestras.

La clase más débil es el 5

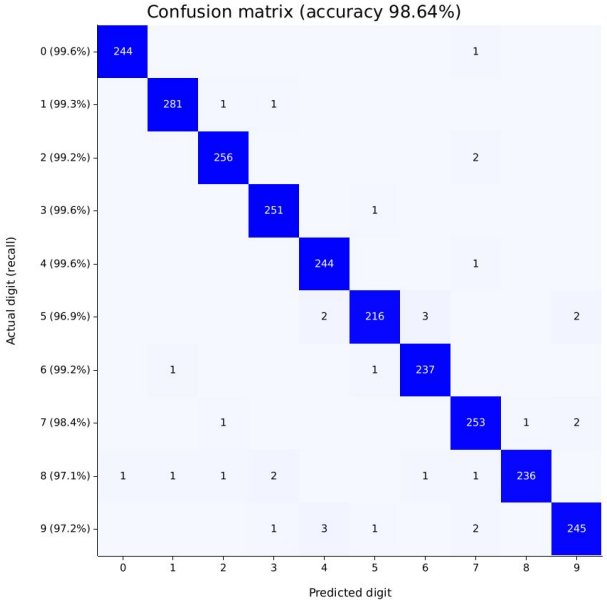
96,9% de recall: se confunde con el 4, el 6 y el 9. Todas las clases quedan entre 96,9% y 99,6%.

Comparación de matrices de confusión

03. Ejercicio 3 · mismo test, antes y después



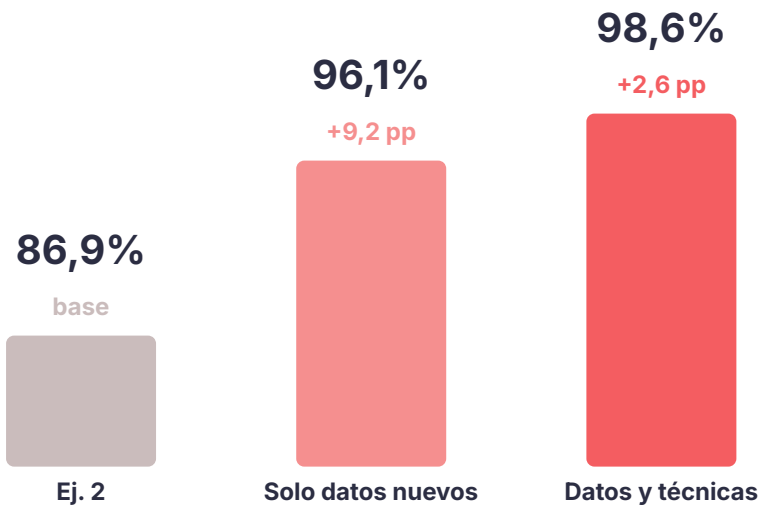
Ejercicio 2 · 86,86%



Ejercicio 3 · 98,64%

Resultado: 98,64% en producción

03. Ejercicio 3 · accuracy en digits_test.csv, eje desde 80%



Recall	Ejercicio 2	Ejercicio 3
Dígito 8	0,0%	97,1%
Dígito 5	88,3%	96,9%
Dígito 9	93,7%	97,2%
Dígito 2	94,6%	99,2%

Objetivo de CompanyX: 98%

Cumplido: 98,64% con loss de 0,047. Todas las clases quedan entre 96,9% y 99,6% de recall.

Métricas: comparación de modelos

03. Ejercicio 3 · validación y test en digits_test.csv

Modelo	Validación	Test	Loss test	Recall mín.
Ej. 2 con digits.csv	96,83%	86,86%	1,645	0,0% (dígito 8)
Ej. 2 con more_digits.csv	95,90%	96,08%	0,129	91,4% (dígito 8)
Ej. 3 con técnicas	98,06%	98,64%	0,047	96,9% (dígito 5)

Qué se compara

Las tres filas se evalúan sobre el mismo test. La loss baja de 1,645 a 0,047 y el peor recall sube de 0% a 96,9%.

Ojo con la validación

La validación de more_digits.csv incluye el 8 y es otra partición, por eso no es comparable con la del Ej. 2 con digits.csv.

Respuestas del Ejercicio 3

03. Ejercicio 3

a) Mejor resultado

98,64%

Red [784, 512, 256, 10] con ReLU, He, Adam 0,001, decaimiento de la tasa, weight decay 1e-4, dropout 0,2 y augmentation. 51 épocas.

b) Técnicas usadas

Augmentation y decaimiento de la tasa

Aportan +1,43 pp y +0,76 pp. Sin augmentation el early stopping corta en la época 6; con ella, la red sigue mejorando hasta la 51.

c) Otros factores

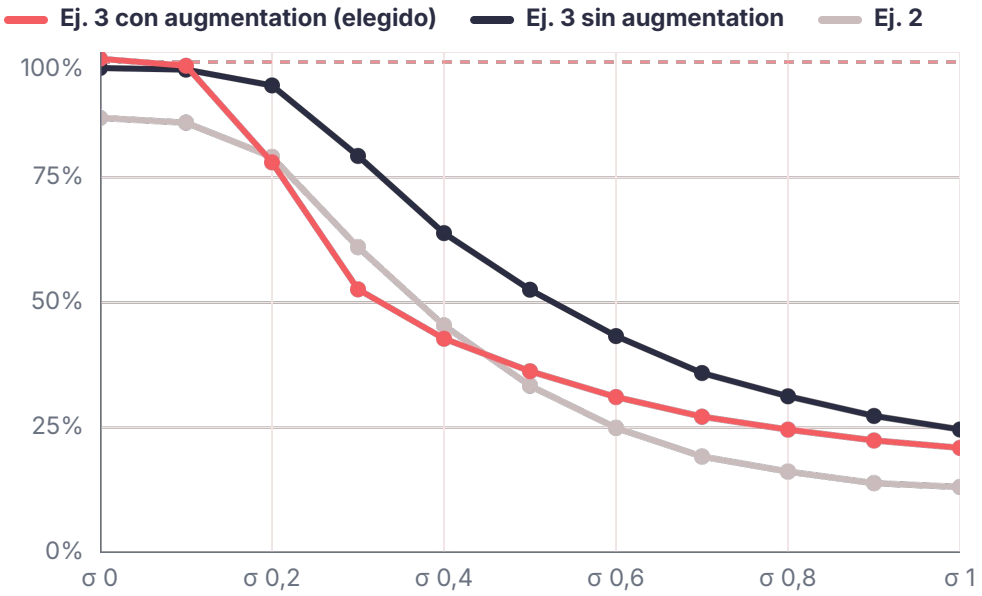
Los datos mismos

Solo por incluir el 8, los mismos hiperparámetros del Ej. 2 pasan de 86,9% a 96,1%. Los datos pesan más (+9,2 pp) que las técnicas (+2,6 pp).

Además: el test está balanceado (unas 250 muestras por clase), así que una clase mal aprendida cuesta cerca de 10% de la accuracy.

Robustez al ruido gaussiano

03. Ejercicio 3 · opcional · accuracy en con x′ = clamp(x + σ·z, 0, 1), misma semilla para los tres



No es robusto

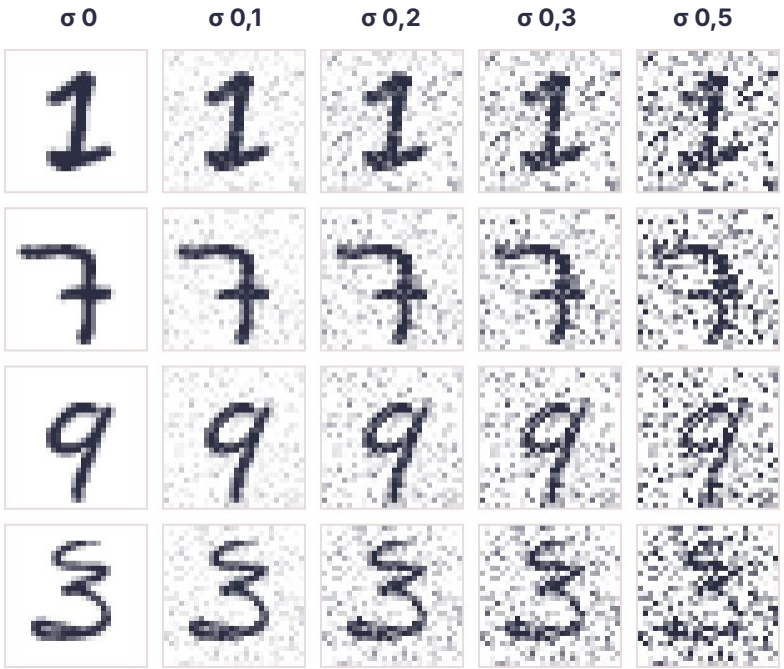
Con σ = 0,1 ya baja del 98% (97,24%) y con σ = 0,2 cae a 77,97%, aunque el dígito sigue siendo claro para una persona.

Sin augmentation aguanta más

Con σ = 0,2 retiene 93,31%. La augmentation geométrica enseña invariancia a traslaciones y rotaciones, no a ruido de píxel.

Qué se rompe primero con el ruido

03. Ejercicio 3 · opcional · modelo elegido



Dígito	σ = 0	σ = 0,2	σ = 0,3
1	99,3%	59,4%	2,1%
7	98,4%	49,4%	10,1%
9	97,2%	46,8%	12,7%
4	99,6%	72,2%	26,9%
0	99,6%	99,2%	94,7%
2	99,2%	99,2%	97,3%

Trazo fino, primero

Los dígitos con poca tinta (1, 7, 9, 4) caen primero; el 0 y el 2 casi no se mueven. El recorte a [0, 1] hace que el ruido sume un velo gris al fondo que el modelo nunca vio, y empuja las predicciones hacia dígitos con mucha tinta.

04. Opcionales

Opcionales

ReLU, features, calibración, ruido e interpretabilidad

Opcionales

01 **ReLU en el perceptrón no lineal**
Ejercicio 1

02 **Features construidas y descartables**
Ejercicio 1

03 **Interpretabilidad: métodos de atribución**
Ejercicios 2 y 3

Calibración (Ejercicio 1) y robustez al ruido (Ejercicio 3) se presentaron dentro de su ejercicio.

ReLU en el perceptrón no lineal

04. Opcionales · Ejercicio 1 · MSE contra el BigModel, con todas las muestras



Misma grilla de tasas de aprendizaje para los tres; cada uno se queda con la mejor. El eje parte de cero.

Por qué se parece al lineal

Con una sola neurona, ReLU es la recta del lineal con la parte negativa recortada a 0. No agrega la forma de S que tiene la probabilidad del BigModel.

¿Underfitting?

Igual que el lineal: R^2 0,766 contra 0,762, y la loss se aplana en el mismo nivel.

¿Saturación?

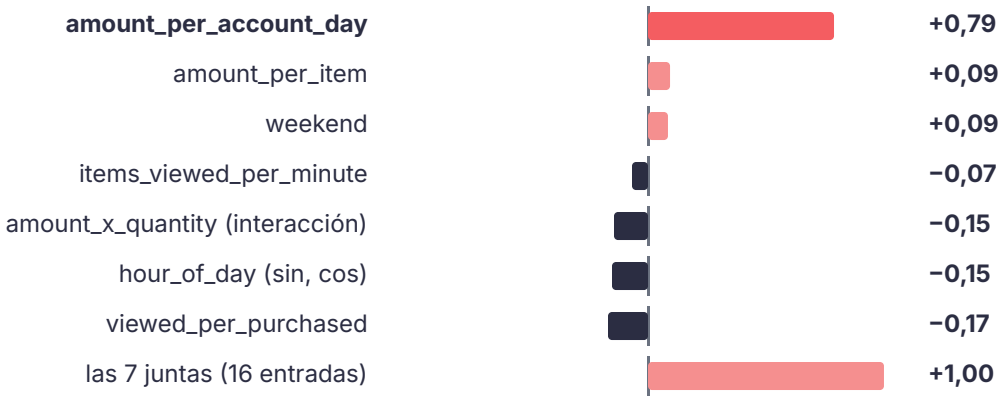
No satura arriba: el 5,5% de sus salidas pasa de 1 (hasta 1,43), así que no es una probabilidad. Abajo, con entrada negativa la salida y el gradiente valen 0.

¿Cambia la elección?

No. El sigmoide sigue siendo el elegido: menor error y salida en [0, 1] por construcción.

Features construidas

04. Opcionales · Ejercicio 1 · ΔF1 de validación cruzada contra la base (F1 0,898 ± 0,007), en pp



Mismo protocolo que el estudio de generalización: mismos 5 folds, scaler por fold y umbral F1 out-of-fold. El test no se consulta.

Solo una ayuda sola

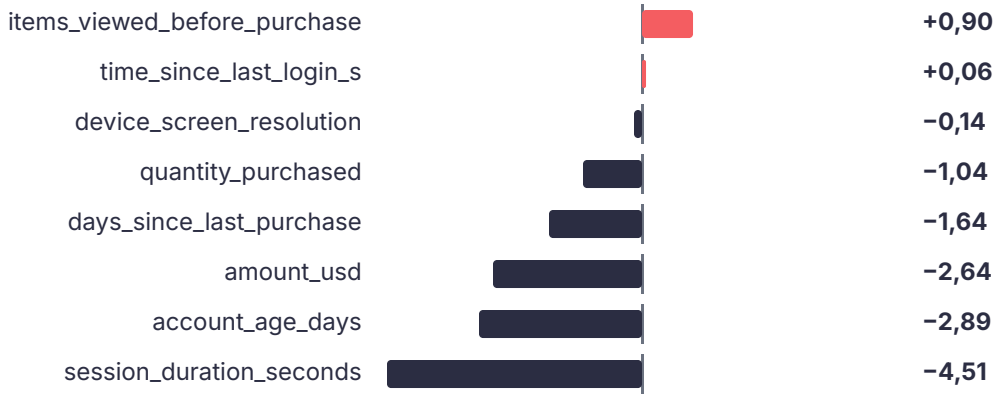
amount / (edad de la cuenta + 1): +0,79 pp de F1. log(edad) es una entrada que la neurona no podía formar, y es la derivada más correlacionada con el fraude (ρ = 0,77).

Correlación no alcanza

Los otros cocientes y la interacción no suman: su información ya es casi combinación lineal de las entradas. Hora y fin de semana no tienen señal.

¿Qué columnas se pueden descartar?

04. Opcionales · Ejercicio 1 · ΔF1 de CV al sacar cada columna, en pp



Se pueden sacar

device_screen_resolution y time_since_last_login_s: no cambian el F1, como anticipaba su correlación casi nula. Sacar items_viewed_before_purchase incluso mejora (+0,90 pp).

Indispensables

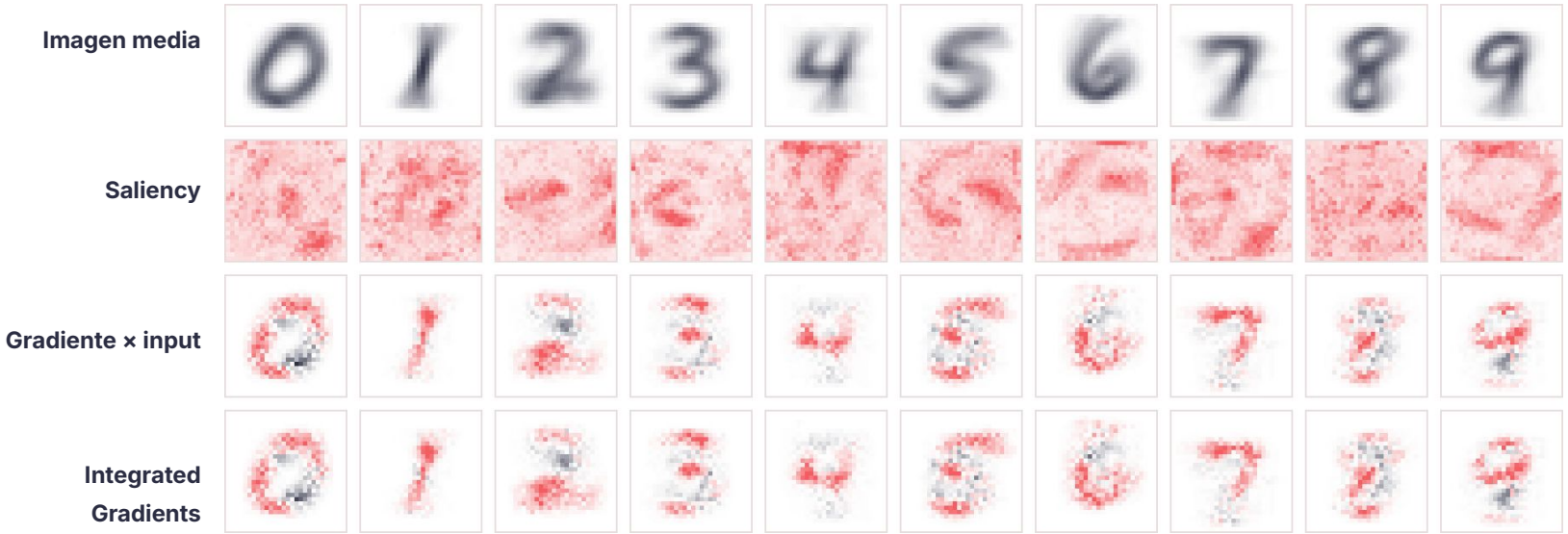
session_duration_seconds, account_age_days y amount_usd: sacarlas cuesta entre 2,6 y 4,5 pp de F1.

Recomendación

Sumar amount_per_account_day y sacar las dos columnas sin señal. El modelo principal no se cambia: en test da F1 0,883 contra 0,880, sin diferencia real.

Interpretabilidad: ¿qué mira la red?

04. Opcionales · Ejercicios 2 y 3 · mapas medios por clase, 100 dígitos de test bien clasificados por clase (modelo del Ej. 3)



Coral: evidencia a favor de la clase. **Azul:** en contra. El 1 se decide por el trazo vertical, el 7 por la barra superior, el 0 por el anillo completo. Saliency es difusa y cae mayormente fuera de la tinta; gradiente × input e IG coinciden.

Atribución de los errores

04. Opcionales · Ejercicios 2 y 3 · qué trazo votó por la clase equivocada

Un 1 predicho como 2



imagen | IG → predicha | IG → verdadera

El pie horizontal del 1 aporta al 2 (parece la base de un 2).

Un 2 predicho como 7



imagen | IG → predicha | IG → verdadera

El trazo superior horizontal es la evidencia del 7.

Un 6 predicho como 5



imagen | IG → predicha | IG → verdadera

La barra superior suma al 5 y resta al 6.

0,37%

error medio de completitud de IG (máximo 4,03%, 1020 casos)

Límites

Con baseline negro, un píxel sin tinta tiene atribución exactamente 0: IG no muestra evidencia por ausencia de tinta. Las atribuciones explican el modelo, no los datos.

Conclusiones

05. Cierre

La capacidad tiene un techo

Una neurona alcanza F1 de 0,880 en fraude gracias al sigmoide y a un buen preprocesamiento; la red multicapa la supera con creces en dígitos.

El dato manda

Una clase ausente fija el techo en cerca de 90%, sin importar los hiperparámetros. Agregar el 8 aportó más que todas las técnicas juntas.

Evaluar bien es la mitad

Test consultado una sola vez, K-Fold y métricas por clase evitaron ver como buenos modelos que no lo eran.

Variar de a un factor

Grillas de un factor a la vez y una escalera acumulativa permiten atribuir cada mejora a una causa concreta.

Ranking no es probabilidad

El TinyModel ordena bien pero no está calibrado. Con Platt, el umbral por costo cuesta 2,6 veces menos que el umbral F1.

La robustez no viene gratis

98,64% en test no implica robustez: con ruido leve cae a 78%. La augmentation geométrica no protege del ruido de píxel.

Muchas gracias